

RELAZIONE TECNICA E DOCUMENTAZIONE

CarRentalHub

Advanced Programming Languages A.A. 2024/25

Francesco Denaro - 1000050594

Piero Galatà - 1000066734

Indice

1.	Introduzione	3
2.	Architettura del sistema	4
2.1.	Comunicazione tra i componenti	4
2.2.	Vantaggi dell'architettura	4
3.	Descrizione componenti	5
3.1.	Frontend [.NET Avalonia]	5
3.1.1.	Funzionalità modulo Frontend	6
3.1.2.	Implementazione	7
3.1.3.	Struttura e Configurazione per Avalonia	18
3.1.4.	Dettagli librerie utilizzate	18
3.2.	Modulo Backend [Go]	19
3.2.1.	Struttura del backend e dettagli implementativi	20
3.2.2.	Sicurezza	24
3.2.3.	Database	24
3.2.4.	Endpoints	24
3.2.5.	Gestione della concorrenza e parallelizzazione	26
3.2.6.	Implementazione della cache	27
3.3.	Statistics Module [Python]	28
3.3.1.	Struttura e dettagli implementativi	29
4.	Avvio applicazione	31
4.1.	Modulo Frontend	31
4.2.	Moduli Backend e Stats	32
4.3.	Avvio del Sistema	32

1. Introduzione

Lo scopo del progetto **CarRentalHub**, sviluppato per il corso *Advanced Programming Languages* (A.A. 2024/2025), è quello di realizzare una piattaforma desktop completa per la gestione dei noleggi di veicoli.

L'applicazione si compone di quattro moduli principali:

- **Frontend (C# + Avalonia):** responsabile dell'interfaccia utente, permette ai clienti e agli amministratori di interagire con il sistema. Include schermate per login, registrazione, visualizzazione dei veicoli, prenotazioni e una sezione mappa per i punti di ritiro. Gli amministratori accedono a una dashboard dedicata per la gestione dei veicoli e delle prenotazioni e il monitoraggio.
- **Backend (Go):** gestisce la logica di business del sistema. Espone API REST per tutte le funzionalità fondamentali: autenticazione, gestione degli utenti, gestione del parco veicoli e delle prenotazioni.
- **Database (PostgreSQL):** archivio centrale dei dati dell'applicazione, memorizza in modo strutturato e sicuro tutte le informazioni su utenti, veicoli e prenotazioni.
- **Modulo Statistiche (Python):** si occupa di analizzare i dati raccolti e generare report statistici e grafici. Questo modulo interagisce con il backend ed è pensato per supportare gli amministratori con informazioni utili a prendere decisioni strategiche.

Il sistema prevede due ruoli principali:

- **Cliente:** può registrarsi, accedere, cercare e prenotare veicoli, consultare la propria cronologia e visualizzare sulla mappa i punti di ritiro.
- **Amministratore:** ha accesso a strumenti di gestione (veicoli, utenti, prenotazioni) e a funzionalità avanzate di monitoraggio tramite report e grafici.

L'architettura del progetto segue il paradigma three-tier, con separazione chiara tra interfaccia utente, logica applicativa e persistenza dei dati. La comunicazione tra i moduli avviene tramite

API REST. Inoltre, è prevista l'integrazione con servizi esterni, come Maps UI, per fornire una visualizzazione geografica dei punti di ritiro.

2. Architettura del sistema

L'architettura di riferimento del sistema **CarRentalHub** è di tipo **three-tier** e prevede una netta separazione tra:

- **Frontend** (interfaccia utente)
- **Backend** (logica applicativa)
- **Data layer** (database e modulo statistico)

L'interazione tra i livelli avviene esclusivamente tramite **API REST**, garantendo modularità e indipendenza dei componenti. Questa architettura garantisce separazione delle responsabilità tra i componenti, flessibilità e scalabilità. I livelli comunicheranno tra loro tramite interfacce ben definite.

2.1. Comunicazione tra i componenti

I principali moduli si interconnettono tra loro in questo modo:

- Il **Frontend** comunica con il **Backend** attraverso chiamate REST HTTP per richiedere dati e inviare operazioni (es. prenotazioni, login, aggiornamenti). Dati che verranno utilizzati per aggiornare la GUI in modo reattivo.
- Il **Backend** è connesso al **database** utilizzando il driver PostgreSQL nativo per Go.
- Quando sono necessarie analisi statistiche, il **Frontend** comunica con il modulo **Python** (che espone anch'esso delle API REST) per ottenere i risultati.

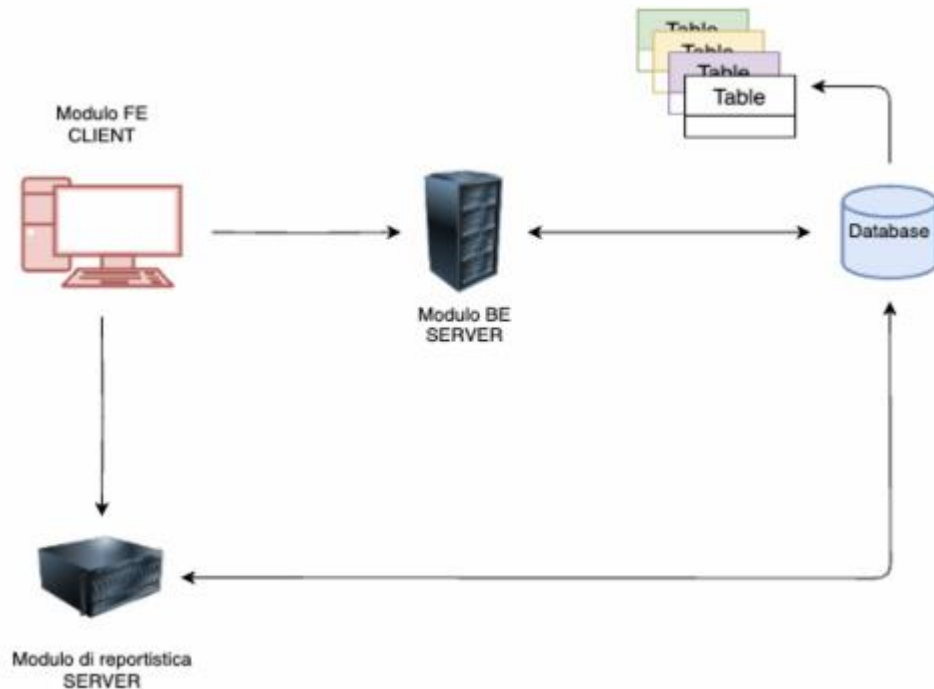
2.2. Vantaggi dell'architettura

Adottando l'architettura descritta, possiamo definire i principali vantaggi come segue:

- **Separazione delle responsabilità:** ogni livello si occupa di un insieme specifico di funzionalità in modo completamente indipendente dagli altri moduli.
- **Manutenibilità:** ogni modulo può essere aggiornato o sostituito in modo indipendente.

- **Scalabilità:** la struttura consente di scalare singoli moduli in base al carico (es. backend o modulo statistiche).
- **Flessibilità:** grazie all'uso di REST API, è possibile estendere il sistema o integrarlo con nuovi moduli o client.

Nella figura di seguito è mostrata l'architettura di riferimento.



3. Descrizione componenti

3.1. Frontend [.NET Avalonia]

Per il modulo frontend dell'applicazione è stato scelto di utilizzare il linguaggio **C#** insieme al framework **Avalonia UI**, ovvero una libreria open source per la creazione di applicazioni desktop multiplatform. La scelta di **C#** e **.NET** si basa sulla robustezza, la flessibilità e l'ampio supporto della community, che permettono di sviluppare applicazioni scalabili e mantenibili. Inoltre, **.NET** è ideale per la creazione di applicazioni multiplatform, grazie alla sua compatibilità con ambienti Windows, macOS e Linux.

Avalonia UI è stato scelto per la sua capacità di integrare l'approccio **MVVM (Model-View-ViewModel)**, che favorisce una chiara separazione tra la logica di presentazione e quella di gestione dei dati. Questo approccio non solo migliora la manutenibilità e la testabilità del codice, ma permette anche di sviluppare funzionalità nuove senza compromettere la struttura esistente dell'applicazione. In particolare, il pattern MVVM consente di implementare facilmente il binding dei dati e la gestione degli eventi in modo dichiarativo, evitando che la logica applicativa interferisca direttamente con l'interfaccia utente (View). In questo modo, il codice risulta più modulare e facilmente estendibile.

Vengono utilizzati diversi **pacchetti NuGet** per aggiungere funzionalità all'applicazione. Di seguito sono elencati alcuni tra i principali utilizzati nel progetto:

- **Avalonia**: il pacchetto base per l'implementazione di interfacce utente desktop multiplatform con Avalonia. Fornisce tutte le funzionalità essenziali per costruire applicazioni desktop, compatibili con Windows, macOS e Linux.
- **Avalonia.Desktop**: estende le funzionalità di **Avalonia** per lo sviluppo di applicazioni desktop, offrendo supporto per finestre, controlli avanzati e ottimizzazione per la gestione delle interfacce utente su piattaforme desktop.
- **Avalonia.Themes.Fluent**: un pacchetto che fornisce una serie di temi grafici predefiniti basati su **Fluent Design**, l'interfaccia visiva moderna di Microsoft, per applicazioni eleganti e coerenti su tutte le piattaforme supportate.
- **CommunityToolkit.Mvvm**: una libreria che semplifica l'implementazione del pattern **MVVM** fornendo classi e funzionalità per la gestione di comandi, notifiche di cambiamento dei dati e binding, migliorando la struttura e la manutenibilità del codice.
- **Mapsui** e **Mapsui.Avalonia**: pacchetti per l'integrazione di mappe interattive nell'applicazione. **Mapsui** è una libreria open source per visualizzare e interagire con dati geografici, mentre **Mapsui.Avalonia** fornisce il supporto per l'uso di questa libreria in applicazioni basate su **Avalonia**.

3.1.1. Funzionalità modulo Frontend

Il modulo Frontend offre un'ampia gamma di funzionalità pensate per facilitare l'interazione degli utenti con il sistema, sia per i **customer** che per gli **amministratori**. Ogni sezione dell'app è progettata per rispondere alle esigenze specifiche di ciascun tipo di utente.

Per i **customer**, l'applicazione fornisce una serie di funzionalità pensate per ottimizzare e soprattutto facilitare il flusso di prenotazione e la gestione delle informazioni personali:

- **Login e Registrazione**: Gli utenti possono creare un account o accedere alla piattaforma inserendo i propri dati attraverso schermate di **login** e **registrazione** semplici e intuitive. Questa sezione garantisce una gestione sicura e rapida dell'accesso.
- **Home Page**: Una volta effettuato l'accesso, l'utente viene indirizzato alla **Home**, dove vengono riassunte le feature caratteristiche dell'app, ovvero le operazioni che un customer potrà effettuare quali prenotazione di un'auto, gestione e ricerca delle proprie

prenotazioni, gestione dati personali e interazione con una mappa che visualizza la posizione degli autosaloni.

- **MyArea:** Gli utenti hanno a disposizione una sezione **MyArea**, un'area personale dove possono consultare e gestire i propri dati. In questa sezione è possibile aggiornare le informazioni personali.
- **Mappa Interattiva:** Grazie all'integrazione di **Google Maps API**, i customer possono visualizzare una **mappa interattiva** che mostra la posizione di tutti gli autosaloni disponibili. Questa funzionalità è particolarmente utile per trovare l'autosalone più vicino e ottenere indicazioni per raggiungerlo, migliorando l'esperienza dell'utente.

Per gli **amministratori**, l'applicazione offre un set di strumenti avanzati per gestire l'intero servizio (oltre al Login all'avvio dell'applicativo):

- **Gestione Auto e Prenotazioni:** Gli amministratori hanno la possibilità di visualizzare e gestire l'intero parco auto suddiviso tra i vari autosaloni. Possono aggiungere nuove auto, modificarne i dettagli, rimuoverle dalla lista o aggiornare la disponibilità. Inoltre, è possibile **monitorare tutte le prenotazioni** effettuate dai customer, con opzioni per modificare lo stato delle prenotazioni, cancellarle o modificarle in base alle necessità.
- **Registrazione nuovi admin:** Solo un amministratore può creare una nuova utenza da admin, ovvero solo dopo aver verificato le proprie credenziali da admin.
- **Grafici e Statistiche:** Un'altra importante funzionalità per gli amministratori è l'accesso a una **sezione di report e statistiche**. Qui possono visualizzare grafici e analisi dettagliate sulle informazioni degli utenti, le prenotazioni e i trend di utilizzo, fornendo dati utili per prendere decisioni strategiche sulla gestione del servizio.

3.1.2. Implementazione

La struttura segue, come detto, l'approccio MVVM. Pertanto la directory principale presenta 3 subfolder (Model, ViewModel, View) per separare le 3 tipologie di file. Inoltre sono presenti altre 4 folder ovvero Services, Helpers, Converters e Assets.

Andando nel dettaglio dei componenti troviamo:

Assets: Contiene le risorse statiche, ovvero immagini e icone, utilizzate nelle varie interfacce utente. Troviamo le 4 immagini visualizzate nella homepage offerta ai customer subito dopo l'accesso e un file che mappa le icon sfruttando la classe StreamGeometry di Avalonia.

Converters: Sono dei file utilizzati per trasformare i dati tra la **View** e il **ViewModel**. Nello specifico è stato realizzato per convertire la presenza di una stringa non vuota in true e viceversa (*VisibilityConverter*). Così gli elementi che presentano testi vuoti (come ad esempio i semplici messaggi di errore) non sono presenti nella struttura di elementi della pagina finché non hanno un contenuto da mostrare. L'altro converter (*IconConverter*) verifica se una determinata iconKey è presente tra quelle definite, altrimenti ritorna una icon di "not found" di default. In pratica, i converters permettendo di mantenere il codice pulito e facilmente testabile.

Services: Questa folder è stata realizzata per generare un'architettura a sé stante con il compito di gestire le chiamate verso gli endpoint esposti dal modulo Backend e dal modulo Stats. Troviamo:

- **VehicleService:**

- *GetAvailableShowrooms(DateTime? startDate, DateTime? endDate):* funzionalità di recupero di showroom e relativi veicoli disponibili in un determinato periodo di tempo;
- *GetAllVehicles():* funzionalità di recupero di tutte le auto presenti nel DB;
- *AddVehicle(VehicleModel vehicle):* funzionalità di aggiunta di un nuovo veicolo;
- *EditVehicle(VehicleModel vehicle):* funzionalità di modifica di un dato veicolo;
- *DeleteVehicle(int VehicleId):* funzionalità di cancellazione di un dato veicolo;

- **UserService:**

- *Authenticate(string username, string password):* funzionalità di accesso dell'utente. Include l'utilizzo di un PasswordHasher e TokenHelper per rispettivamente effettuare l'hash della password prima di trasmetterla nella comunicazione con il Backend e registrare il token richiesto per l'utilizzo di tutte le altre funzionalità.
- *Register(string email, string username, string password, bool isAdmin, string region):* permette la registrazione di un nuovo user, discriminando tra quando l'user è un customer e quando un amministratore di sistema.
- *EditProfile(UserModel User, string? NewPassword):* funzionalità di aggiornamento dei dati personali di un utente (*customer*)

- **BookingService:**

- *AddBooking(int vehicleId, DateTime? startDate, DateTime? endDate):* funzionalità di nuova prenotazione di un veicolo
- *GetAllBookings():* funzionalità di recupero di tutte le prenotazioni presenti a sistema (sarà il modulo Backend a determinare se l'utente è un admin e quindi potrà avere accesso a tutte le prenotazioni, oppure è un customer e quindi visualizzerà solo le sue prenotazioni)
- *DeleteBooking(int BookingId):* funzionalità di cancellazione di una data prenotazione
- *EditBooking(BookingModel booking, int vehicleId):* funzionalità che permette di aggiornare alcuni campi di una prenotazione

- **HttpService:**

Questo file definisce un servizio HTTP in C# che fornisce metodi per eseguire richieste HTTP sincrone (POST, PUT, DELETE, GET). Utilizza un'istanza statica di *HttpClient* per inviare le richieste e gestire la serializzazione e deserializzazione dei dati in formato JSON. I metodi sono generici e restituiscono una risposta deserializzata in un tipo specificato (*TResponse*, definito dalla funzione che richiama il metodo HTTP). Se disponibile, il servizio aggiunge un token di autorizzazione Bearer nell'header della

richiesta. Il tutto è gestito in modo sincrono, con gestione delle eccezioni per errori di connessione e altri problemi durante l'esecuzione della richiesta.

Helpers: Contiene i file helpers utilizzati all'interno del codice, nello specifico **TokenHelper.cs** e **PasswordHasher.cs**.

- PasswordHasher.cs: fornisce la funzionalità di Hash utilizzata per cryptare le password degli utenti così da non inviarle in chiaro
- TokenHelpers.cs: permette di memorizzare/cancellare il token restituito dal Backend al momento dell'accesso e anche la funzione di recupero per includere il token tra gli headers delle varie richieste effettuate.

Models: rappresentano la struttura dei dati utilizzati nell'applicazione. Definiscono quindi gli oggetti che contengono le informazioni da visualizzare nell'interfaccia utente. Vengono usati sia per definire la tipologia di serializzazione sulle response delle API e sia per ottimizzare e facilitare l'utilizzo e la visualizzazione sulla UI dei dati.

- **UserModel** modella i dati relativi ad un'entità user:
 - *string Username,*
 - *string Email,*
 - *string Password,*
 - *string Region,*
 - *string Role*
- **BookingModel:** modella i dati relativi ad una prenotazione:
 - *int Id,*
 - *int UserId,*
 - *string Username,*
 - *int vehicleId,*
 - *string VehicleModel,*
 - *float TotalPrice,*
 - *DateTime StartDate,*
 - *DateTime EndDate,*
 - *DateTime CreatedAt*
- **VehicleModel:** modella i dati relativi ad un veicolo:
 - *int Id,*
 - *string Model,*
 - *string Category,*
 - *int CarShowroomID,*
 - *string ShowroomName*
- **ShowroomModel:** modella i dati relativi ad un autosalone:
 - *int Id,*
 - *string Name,*
 - *string Location,*
 - *float Latitude,*
 - *float Longitude,*

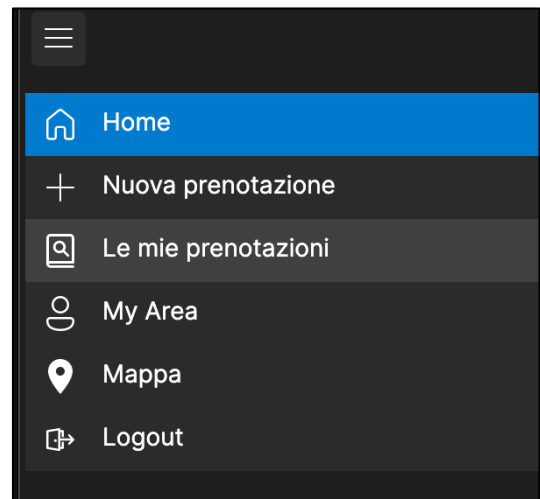
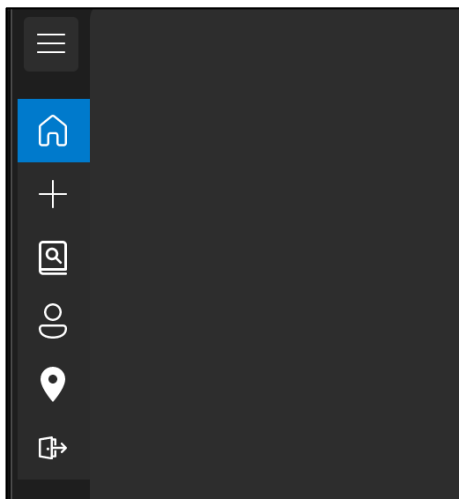
- **List<VehicleModel> Vehicles**
 - **Datetime CreatedAt**
- **ListItemTemplate:** Rappresenta una modello utilizzare per creare la lista di componenti del menu laterale dell'app (sia per customer che per admin):
 - **Func<ViewModelBase> CreateInstance,**
 - **string IconKey,**
 - **string Label**

View: contiene i file che definiscono l'interfaccia utente dell'app. Questi file, scritti in AXAML, descrivono l'aspetto visivo dell'applicazione, come finestre, controlli e layout. AXAML (Avalonia XAML) è il linguaggio di markup usato per Avalonia. Basato su XML, è utilizzato per definire l'interfaccia utente in modo dichiarativo.

Ogni vista è legata al suo **ViewModel**, trattato di seguito, il quale gestisce la logica e i dati, mantenendo separati i due aspetti. Grazie a questo legame tramite il **data binding**, i cambiamenti nei dati vengono automaticamente riflessi nell'interfaccia utente.

I file sono i seguenti:

- **MainWindow.axaml:** è il file che definisce la window principale che farà da contenitore per tutti gli altri file definiti come UserControl.
- **HomeView.axaml:** rappresenta la prima vista che viene mostrata all'avvio, offrendo la possibilità di scegliere se effettuare l'accesso e registrare una nuova utenza.
- **LoginView.axaml:** Scegliendo di effettuare l'accesso viene mostrato il form per inserire username e password.
- **RegisterView.axaml:** Scegliendo la registrazione viene mostrato un form per inserire tutti i dati della nuova utenza.
N.B.: La nuova utenza è un nuovo customer (NON admin)
- **UserMainView.axaml:** Questo file definisce la struttura principale della vista per un utente loggato. Viene utilizzata una SplitView con un Pane e un Content. Il primo permetterà di visualizzare un menu laterale con Icon e Label, come sopra descritto in riferimento al *ListItemTemplate*, mentre il secondo è il contenuto relativo alla voce del menu selezionata.



- **UserHomePageView.axaml.:** Questa è la pagina di default visualizzata dal customer, ovvero la sua homepage, da cui è già direttamente possibile accedere alle altre voci del menu quali “Nuova prenotazione”, “Le mie prenotazioni”, “MyArea” e “Mappa”.
- **NewBookingsView.axaml:** Da qui è possibile prenotare una nuova auto. Viene data infatti la possibilità di inserire il periodo di interesse per mostrare la lista di autosaloni con auto disponibili in quel periodo. Una volta selezionato l’autosalone verranno mostrati i veicoli disponibili. Selezionando il veicolo si potrà completare la prenotazione.
 - Step 1: Scegli date

Nuova Prenotazione

Per effettuare una prenotazione, inizia selezionando le date in cui desideri noleggiare un'auto. Scelto il periodo ti mostreremo gli autosaloni disponibili, seleziona quello che ti interessa. Seleziona il veicolo che preferisci tra quelli che vedi presenti in lista e completa la prenotazione in pochissimi passaggi!

Scegli la data per la tua prenotazione

02/04/2025 27/04/2025 ⓘ

Vedi gli autosaloni disponibili

- Step 2: Seleziona autosalone

Nuova Prenotazione

Per effettuare una prenotazione, inizia selezionando le date in cui desideri noleggiare un'auto. Scelto il periodo ti mostreremo gli autosaloni disponibili, seleziona quello che ti interessa. Seleziona il veicolo che preferisci tra quelli che vedi presenti in lista e completa la prenotazione in pochissimi passaggi!

Cerca per comune

Cerca Mostra tutte

Nome	Comune	Q.tà veicoli
CarShowroom RM	Roma	2
CarShowroom FI	Firenze	1
CarShowroom PA	Palermo	3
CarShowroom TO	Torino	2
CarShowroom VR	Verona	2

< >

Vedi auto disponibili Indietro

- Step 3: Seleziona veicolo

Nuova Prenotazione

Per effettuare una prenotazione, inizia selezionando le date in cui desideri noleggiare un'auto. Scelto il periodo ti mostreremo gli autosaloni disponibili, seleziona quello che ti interessa. Seleziona il veicolo che preferisci tra quelli che vedi presenti in lista e completa la prenotazione in pochissimi passaggi!

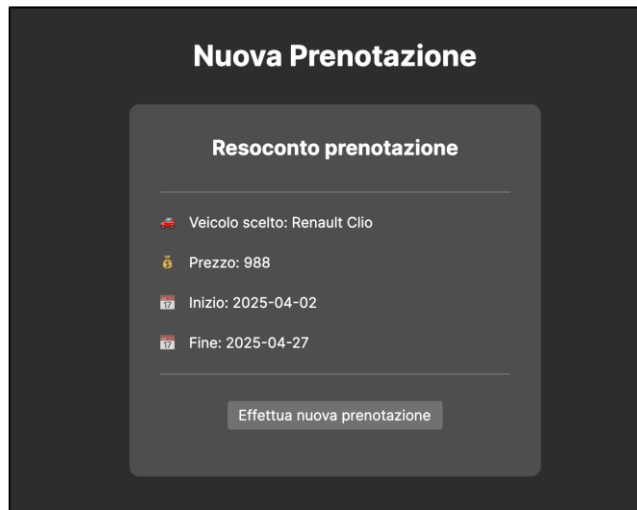
Inserisci modello o marca

Cerca Mostra tutte

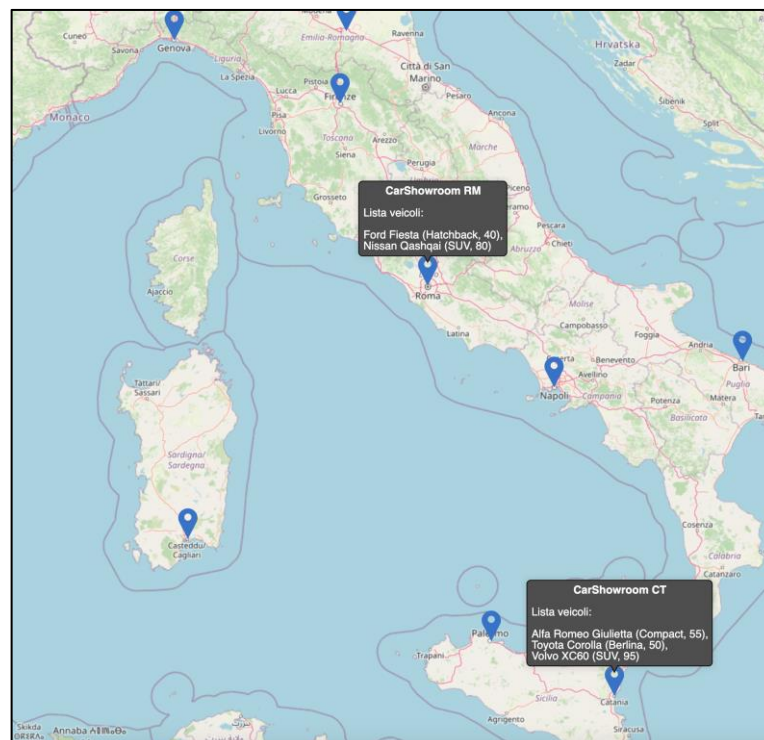
ID	Model	Category	Price
12	Renault Clio	Hatchback	38
22	Kia Sportage	SUV	82

Prenota auto Indietro

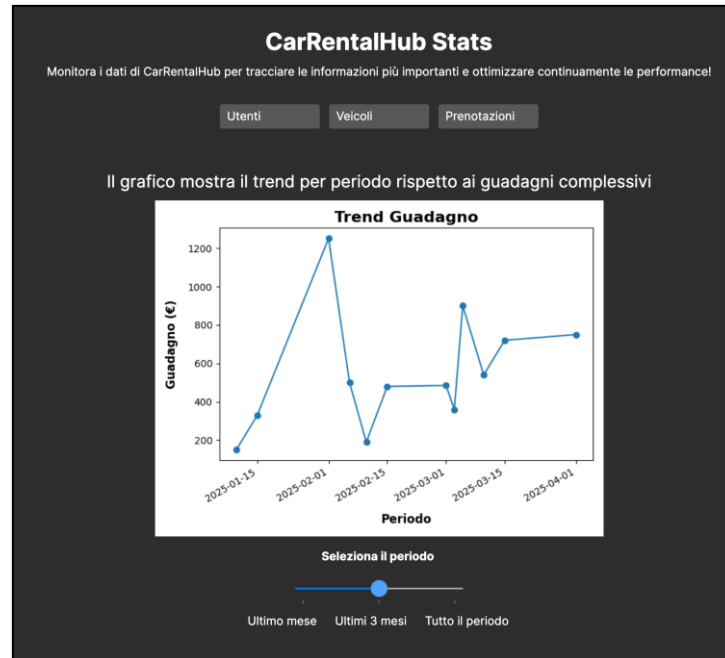
- Step 4: Controlla il resoconto



- **MyBookingsView.axaml:** Nelle Mie Prenotazione l'utente può controllare tutte le sue prenotazioni sia passate che future. Può sia effettuare una ricerca testuale per modello di veicolo sia filtrare per range temporale, così da mostrare solo le prenotazioni in quel periodo.
- **MyAreaView.axaml:** Nella MyArea l'utente può richiedere la modifica dei suoi dati personali quali Username, Email e Regione. Potrà inoltre modificare la password, solo dopo aver inserito anche la password attuale.
- **TileMapView.axaml:** Attraverso l'utilizzo della libreria MapsUI, viene mostrata una mappa interattiva, con una serie di marker per ognuno degli showroom. L'inizializzazione e le interazioni con la mappa sono gestite nel ViewModel, mentre nella View viene solo dichiarato il MapControl.



- **AdminBookingView.axaml:** Rappresenta la pagina di gestione delle prenotazioni lato Admin. Qui l'amministratore potrà ricercare, modificare o cancellare le prenotazioni, avendo pieno controllo delle informazioni.
- **AdminVehicleView.axaml:** Viene presentato l'elenco di tutti i veicoli presenti nel parco auto, con la possibilità di gestirli, aggiungendone di nuovi oppure modificando o eliminando altri veicoli.
- **StatsView.axaml:** Questa è l'area in cui l'amministratore monitora le metriche dell'applicazione, suddivise in tre categorie: utenti, veicoli e prenotazioni. Per gli utenti, visualizza i più attivi, la provenienza geografica e consente di ottenere informazioni dettagliate tramite ID. Per i veicoli, mostra la disponibilità in percentuale e i più ricercati. Nella sezione prenotazioni, evidenzia l'andamento dei guadagni e il numero di prenotazioni nel tempo, con un grafico dei veicoli più redditizi. Questi ultimi dati possono essere filtrati per periodo: ultimo mese, ultimi tre mesi o periodo completo.



ViewModels: rappresentano la logica dell'interfaccia utente, fungendo da intermediari tra la vista e il modello, gestendo i dati e i comandi necessari per aggiornare l'interfaccia in modo reattivo. Di seguito l'elenco:

- **PaginatedViewModel.cs:** è una classe astratta progettata per gestire la paginazione dei dati. Definisce un insieme di proprietà e metodi per visualizzare elenchi di oggetti in modo paginato sfruttando il meccanismo di tipo generico. La proprietà *PageSize* determina il numero di elementi per pagina, mentre *CurrentPage*, *IsPreviousPageEnabled* e *IsNextPageEnabled* gestiscono la navigazione tra le pagine. La lista completa degli elementi è mantenuta in *_allItems*, mentre gli elementi della pagina corrente sono memorizzati in *_items*. Il metodo *LoadItems* carica tutti gli elementi, mentre *UpdatePaginatedItems* aggiorna l'elenco visualizzato in base alla pagina corrente e alla presenza di risultati. La gestione della visibilità dei pulsanti di navigazione garantisce che non vengano mostrati se non necessari. Chi eredita questa classe dovrà solo definire il tipo degli Items e implementare il metodo astratto *LoadAllItems()* il quale dovrà solo ritornare una List di items.
- **SearchableViewModel.cs:** Estende **PaginatedViewModel**, aggiungendo le funzionalità di filtraggio dei dati in base alla ricerca. Gestisce una query di ricerca (*SearchQuery*) e applica il filtro ai dati usando il metodo *ApplySearch()*. La ricerca può essere estesa per includere altre tipologie di filtro, come la ricerca per ID (int) con il metodo *ApplySearchById*. Inoltre integra la logica di paginazione con la ricerca, permettendo di visualizzare solo i risultati filtrati nella pagina corrente. Con comandi come *SearchItems*, *GoToPreviousPage* e *GoToNextPage*, l'utente può navigare tra i risultati filtrati, rendendo l'interazione con i dati più dinamica. La possibilità di ripristinare la paginazione e visualizzare tutti i risultati consente un'esperienza utente fluida e personalizzabile. Utilizzando questa struttura, altri ViewModel possono facilmente implementare sia la paginazione che la ricerca riutilizzando il codice della classe **SearchableViewModel**, semplificando la gestione dei dati.
- **AdminBookingViewModel.cs:** Estende la classe **SearchableViewModel<BookingModel>** e implementa la logica per gestire la visualizzazione, modifica ed eliminazione delle prenotazioni nell'interfaccia amministrativa. Grazie all'ereditarietà da **SearchableViewModel**, la classe supporta funzionalità di ricerca e paginazione degli elementi. La gestione delle prenotazioni comprende la visualizzazione dei dettagli di una prenotazione selezionata, la possibilità di modificare le date di inizio e fine, nonché di scegliere un veicolo disponibile in base alla data di prenotazione. Inoltre, permette di visualizzare un form di modifica e di eliminare una prenotazione, previa conferma dell'admin. La logica di interazione con l'interfaccia è basata sui comandi MVVM (*RelayCommand*), che attivano le azioni di modifica, salvataggio ed eliminazione, aggiornando di conseguenza l'interfaccia con messaggi di errore o successo. Il caricamento dei dati avviene tramite l'implementazione del metodo *LoadAllItems()* ereditato da **PaginatedViewModel**.
- **AdminVehicleViewModel.cs:** estende la classe **SearchableViewModel<VehicleModel>** e gestisce le operazioni amministrative sui veicoli all'interno dell'applicazione. Consente di visualizzare, aggiungere, modificare ed eliminare i veicoli tramite un'interfaccia utente

basata sul pattern MVVM.

La classe fornisce le funzionalità di ricerca ereditate, grazie all'implementazione del metodo *ApplySearch()*, che filtra i veicoli in base al modello o alla categoria.

L'interfaccia utente è suddivisa in una lista di veicoli visibili e un form di gestione dei veicoli, che può essere attivato per aggiungere o modificare i veicoli.

L'azione di aggiunta o modifica di un veicolo avviene tramite la proprietà *EditingVehicle*, che rappresenta il veicolo che l'amministratore sta creando o modificando. Un aspetto importante della classe è la gestione dei veicoli in base alla loro disponibilità in showroom. Il codice consente di selezionare uno showroom da una lista (memorizzata nella proprietà *ShowroomsList*) e associare un veicolo a uno showroom specifico tramite la proprietà *SelectedShowroom*. Le operazioni di aggiunta e modifica vengono completate tramite i metodi *AddVehicle()* e *EditVehicle()* del servizio **VehicleService**.

- **MyAreaViewModel.cs**: gestisce l'area personale dell'utente, permettendo di modificare dati come nome utente, email, regione e password. Quando l'utente accede, i suoi dati originali sono visualizzati e può modificarli tramite un'interfaccia dedicata. La classe include validazioni per assicurarsi che tutti i campi obbligatori siano compilati prima di salvare le modifiche e consente di modificare la password solo se necessario. Se i dati sono stati modificati, vengono inviati tramite il servizio *UserService.EditProfile()*, che restituisce un messaggio di successo o errore.
- **HomeViewModel.cs**: gestisce la navigazione tra la schermata principale, il login e la registrazione. Utilizza il **MainWindowViewModel** per cambiare il ViewModel corrente, passando a *LoginViewModel* o *RegisterViewModel* a seconda dell'azione dell'utente.
- **MainWindowViewModel.cs**: gestisce la navigazione tra le schermate principali (*HomeView*, *LoginView*, *RegisterView* e *UserMainView*). Contiene la proprietà *CurrentView* che tiene traccia della ViewModel attualmente visibile, inizializzandola con **HomeViewModel** come schermata di partenza. Il metodo *ChangeViewModel()* permette di cambiare la schermata visualizzata, aggiornando la ViewModel corrente.
- **LoginViewModel.cs**: gestisce la logica del login dell'utente. Contiene le proprietà per l'email/username, la password e l'eventuale messaggio d'errore. Quando l'utente invia i dati di login tramite il comando *SubmitLogin()*, la ViewModel verifica se i campi sono validi. Se i campi sono vuoti, viene mostrato un errore, altrimenti, viene chiamato il servizio di autenticazione (*UserService.Authenticate()*). Se il login ha successo, viene mostrata la **UserMainViewModel**, che cambierà dinamicamente a seconda che l'utente sia un customer o un admin. Se il login fallisce, viene mostrato un messaggio di errore. Il comando *Back* permette di tornare alla schermata iniziale (*HomeViewModel*).
- **RegisterViewModel.cs**: gestisce il processo di registrazione di nuovi utenti o amministratori. Contiene proprietà per raccogliere i dati inseriti dall'utente, come email, username, password e la regione di appartenenza. Inoltre, la visibilità di vari elementi dell'interfaccia viene gestita dinamicamente tramite delle proprietà booleane, che controllano, ad esempio, se il form di registrazione è visibile o se il messaggio di errore deve essere mostrato. Quando l'utente effettua il submit del form tramite il comando *SubmitRegistration()*, la ViewModel verifica che tutti i campi siano correttamente compilati. Se ci sono errori, come campi vuoti, un messaggio di errore viene visualizzato.

In caso di successo, viene mostrato un messaggio di conferma.

Se l'utente sta registrando un amministratore, il messaggio di successo sarà diverso rispetto a quello di un utente normale. Dopo una registrazione corretta, per un utente normale, viene mostrata una CTA che riporta al form di login così da effettuare l'accesso, mentre solo un messaggio di successo se l'utente è un admin. Il comando *GoToLogin()* consente di passare alla schermata di login, mentre il comando *Back()* permette di tornare alla schermata principale. La logica di navigazione tra le diverse schermate è gestita dalla ViewModel principale, che cambia la vista corrente in base alle azioni dell'utente.

- **UserMainViewModel.cs:** è la ViewModel principale per l'interfaccia dell'utente una volta loggato, gestendo la navigazione tra diverse sezioni dell'applicazione, come la gestione delle prenotazioni e delle aree personali. La logica di questa classe si basa sulla visibilità di una barra laterale (pane), che può essere aperta o chiusa tramite il comando *TriggerPane()*. La proprietà *CurrentPage* rappresenta la vista attualmente visualizzata, che cambia in base alla selezione di un elemento nel menu laterale.
Il menu laterale è popolato con una lista di voci, definite tramite un costruttore che distingue tra utenti normali e amministratori, presentando un insieme diverso di opzioni in base al ruolo dell'utente. Ogni voce del menu è rappresentata da un oggetto **ListItemTemplate**, che definisce un'azione (la creazione di un nuovo ViewModel relativo alla voce selezionata), l'Id di un'icon e una label visibile nel menu.
La selezione di un elemento del menu (tramite la proprietà *SelectedItem*) attiva il cambiamento della vista corrente. Se l'elemento selezionato è il "Logout", viene eseguito il logout dell'utente, rimuovendo il token di autenticazione e tornando alla schermata principale tramite il comando *Logout()*. In caso contrario, viene creato un nuovo ViewModel associato alla voce selezionata e impostato come vista corrente.
- **UserHomePageViewModel.cs:** gestisce la navigazione dell'utente cliente nella home page, permettendo di accedere alle sezioni "Le mie prenotazioni", "Nuova prenotazione", "My Area" e "Mappa". Ogni sezione è associata a un comando (es. *SelectBookingCommand*), che, quando eseguito, cambia la vista selezionando la sezione desiderata.
- **NewBookingViewModel.axaml:** gestisce il processo di prenotazione di un veicolo. Offre funzionalità per la ricerca tra le showroom mostrate e veicoli in base alle date selezionate, e la visualizzazione di un riepilogo della prenotazione.
Inizia con la selezione delle date tramite la ricerca di showroom, che deve validare che le date siano corrette (non nel passato, data di fine non prima della data di inizio). Se i dati sono validi, mostra la lista degli showroom disponibili. Successivamente, scelto lo l'autosalone di intere, l'utente seleziona un veicolo disponibile e può procedere con la prenotazione, che viene aggiunta tramite il servizio di prenotazione. Una volta effettuata la prenotazione, viene visualizzato un riepilogo e la pagina di ricerca viene nascosta.
In caso di errori, vengono visualizzati messaggi di errore pertinenti, come la mancata selezione di un veicolo o la non disponibilità di veicoli per le date indicate. L'utente ha anche la possibilità di tornare indietro e modificare le selezioni.

- MyBookingsViewModel.cs:** gestisce la visualizzazione delle prenotazioni di un utente e la funzionalità di ricerca. Consente agli utenti di cercare le proprie prenotazioni filtrando per il modello del veicolo o per un intervallo di date. Le funzioni di ricerca e di paginazione sono sempre ereditate dalle classi principali **SearchableViewModel** e **PaginatedViewModel**.

Per quanto riguarda il caricamento delle prenotazioni, se l'utente non ha prenotazioni salvate, viene visualizzato un messaggio che lo informa che non ci sono prenotazioni. Se, invece, ci sono prenotazioni, queste vengono caricate e mostrate nella lista. La ricerca per data permette all'utente di filtrare le prenotazioni in base a un intervallo temporale specificato. Se non ci sono prenotazioni che corrispondono ai parametri di ricerca, viene mostrato un messaggio che invita a cambiare i parametri.

In aggiunta, la ricerca per modello del veicolo consente di filtrare le prenotazioni in base al veicolo selezionato. La vista si aggiorna dinamicamente a seconda dei risultati della ricerca, e nel caso in cui non ci siano risultati o prenotazioni, vengono mostrati messaggi di errore appropriati per guidare l'utente a modificare la ricerca o i parametri inseriti.
- StatsViewModel.cs:** gestisce la visualizzazione delle statistiche e dei dati relativi all'applicazione, come le performance degli utenti, la disponibilità dei veicoli, le prenotazioni e i guadagni. La logica di filtraggio delle informazioni viene gestita attraverso comandi di selezione, che permettono di visualizzare statistiche per periodo specifico o per altre categorie, come gli utenti più attivi o i veicoli più richiesti. Gli utenti possono scegliere un intervallo di date tramite una selezione dinamica, che imposta il periodo di riferimento per le statistiche. Selezionando vari filtri (come "Disponibilità veicoli" o "Trend Guadagni"), i dati vengono caricati in tempo reale tramite richieste http verso il modulo **Stats**, e il risultato viene mostrato come immagine o testo. In caso di errori, un messaggio di errore viene visualizzato per l'utente. Inoltre, è possibile visualizzare informazioni dettagliate sugli utenti, come il numero di prenotazioni e l'importo totale speso, utilizzando un ID utente specifico. Il sistema consente anche di modificare dinamicamente il comando per il filtraggio e di gestire l'interfaccia in modo reattivo in base ai comandi e alle selezioni fatte dall'utente.
- TileMapViewModel.cs:** gestisce una mappa interattiva utilizzando la libreria Mapsui. Il suo compito principale è visualizzare una mappa, aggiungere marker (punti di interesse) e gestire l'interazione dell'utente con questi marker.

Inizialmente, viene creata una mappa vuota (MapView) con il metodo *SetupMap()* che gestisce tutta la configurazione iniziale della mappa, compreso l'aggiunta dei livelli e la gestione dell'evento di interazione. Poi il metodo *CreateMarkerLayer()* crea e gestisce il livello dei marker, assegnando uno stile personalizzato a ciascun marker. I marker rappresentano le posizioni degli showroom presenti nel Database, che vengono recuperati tramite il servizio *VehicleService.GetAvailableShowrooms()*.

Ogni showroom è rappresentato quindi da un marker che ha associata una finestra di dettaglio (un callout) che può essere visualizzata quando l'utente interagisce con il marker. Questo callout contiene il nome dello showroom e un elenco dei veicoli disponibili in quel showroom con dettagli quali modello, categoria e prezzo. La finestra del callout è progettata per essere visibile solo quando l'utente interagisce con il marker, e il suo aspetto (font, dimensioni, colori) è personalizzabile tramite il *CalloutStyle*.

Quando un marker viene cliccato, l'evento *MapInfo* viene gestito dalla funzione *MapOnInfo()*, che alterna lo stato di visibilità del callout associato.

3.1.3. Struttura e Configurazione per Avalonia

I file **App.axaml** e **Program.cs** sono fondamentali per configurare e avviare un'applicazione **Avalonia**. **App.axaml** gestisce temi, stili e la navigazione tra viste e ViewModel tramite il **ViewLocator**, mentre **Program.cs** avvia l'applicazione, configurando il framework e determinando la piattaforma di esecuzione. Insieme, permettono una gestione centralizzata e dinamica dell'interfaccia utente e della logica di navigazione.

App.axaml è il file, come detto, che contiene tutte le impostazioni relative ai temi, stili e risorse globali dell'applicazione. All'interno di questo file, viene specificato l'uso di un tema, in questo caso il tema scuro tramite la proprietà **RequestedThemeVariant="Dark"**. Inoltre, vengono definiti vari stili globali per i controlli comuni dell'interfaccia utente come **TextBlock**, **TextBox**, **Button** e **ListBoxItem**. Questi stili personalizzano l'aspetto dell'applicazione, come i colori dei testi, la selezione, il comportamento al passaggio del mouse e l'aspetto degli elementi selezionati.

Nel blocco `<Application.DataTemplates>`, viene configurato un **ViewLocator**, che si occupa di mappare automaticamente le viste ai rispettivi ViewModel. Questo pattern è molto utile per applicazioni MVVM dove il ViewLocator permette di associare in modo dinamico una vista a un ViewModel senza dover scrivere codice manuale per ciascun collegamento tra View e ViewModel.

Il **ViewLocator** ha la funzione principale di associare dinamicamente le viste ai rispettivi ViewModel. Questo viene fatto utilizzando il nome completo del tipo del ViewModel. Il Build del ViewLocator esegue una logica semplice: dato un parametro (che rappresenta il ViewModel), il nome del tipo viene manipolato sostituendo la parola "ViewModel" con "View" per ottenere il nome della vista corrispondente. Se la vista esiste, viene creata e restituita, altrimenti viene visualizzato un TextBlock con un messaggio di errore che indica che la vista non è stata trovata. Inoltre, il Match verifica se il parametro passato è un'istanza di ViewModelBase. Questo è essenziale per determinare se il ViewLocator deve gestire il collegamento tra il ViewModel e la vista, permettendo così una gestione automatica della navigazione dell'interfaccia utente basata sui ViewModel.

Program.cs rappresenta il punto di ingresso dell'applicazione e contiene la logica necessaria per avviare il framework Avalonia. Il metodo *BuildAvaloniaApp()* è utilizzato per configurare il framework Avalonia, specificando l'applicazione da avviare (App) e le opzioni relative alla piattaforma (ad esempio, il rilevamento automatico della piattaforma). Viene anche configurato un log di traccia per il debug, e il metodo *StartWithClassicDesktopLifetime()* avvia l'applicazione in modalità desktop tradizionale, per Windows, macOS e Linux.

3.1.4. Dettagli librerie utilizzate

CommunityToolkit.Mvvm è una libreria parte del Community Toolkit di Microsoft, progettata per semplificare la creazione di applicazioni MVVM in .NET.

Essa offre una serie di strumenti che semplificano la gestione dello stato e la comunicazione tra la UI e i ViewModel, riducendo la quantità di codice boilerplate necessario. Tra le sue caratteristiche principali vi sono gli attributi come *ObservableProperty*, che automatizzano la

creazione delle proprietà osservabili, e comandi come *RelayCommand*, che semplificano la gestione degli eventi e delle azioni dell'utente, senza la necessità di implementare interfacce come *ICommand*. Questo approccio rende lo sviluppo più rapido e meno propenso a errori, migliorando la leggibilità e la manutenzione del codice.

Nell'applicazione, abbiamo sfruttato *CommunityToolkit.Mvvm* per gestire la logica di interazione tra la UI e i dati.

MapsUI, d'altra parte, è una libreria per .NET che consente di integrare mappe interattive nelle applicazioni, utilizzando una vasta gamma di mappe da diverse fonti, come *OpenStreetMap*, *Bing Maps*, e molte altre. *MapsUI* permette di visualizzare dati geospaziali, aggiungere marker, tracciare rotte, e personalizzare l'aspetto delle mappe.

Nell'applicazione, **MapsUI** è stata utilizzata per visualizzare le posizioni degli showroom su una mappa, integrando il sistema con le informazioni geografiche provenienti dai dati delle concessionarie di veicoli. È stato creato un layer personalizzato per i marker, che mostra la posizione degli autosaloni e un callout che visualizza informazioni dettagliate quando l'utente interagisce con un marker. La mappa è stata configurata per centrare la vista su una posizione specifica e consentire la navigazione tra i diversi marker delle concessionarie, offrendo un'esperienza visiva dinamica e interattiva.

3.2. Modulo Backend [Go]

Il backend rappresenta il cuore dell'applicazione **CarRentalHub** ovvero il server principale con il quale si interfacerà il Frontend. È sviluppato in linguaggio **Go** utilizzando la versione 1.21 ed è organizzato secondo un'architettura modulare basata sul pattern **controller-service-repository**. Viene utilizzato il framework web **Gin** per la gestione delle API REST, in quanto fornisce un'ottima combinazione tra prestazioni elevate e semplicità di utilizzo. Gin è un web framework scritto in Go che permette di creare API RESTful in modo rapido e strutturato, supportando routing con parametri, middleware personalizzati e gestione semplificata delle richieste e delle risposte in formato JSON. Nel progetto *CarRentalHub*, Gin viene utilizzato per:

- **Definire le route** e gestire i vari endpoint dell'applicazione;
- **Applicare middleware** per funzionalità trasversali come l'autenticazione JWT e il logging delle richieste;

Gin semplifica anche la **gestione degli errori**, offrendo meccanismi chiari per restituire messaggi strutturati e codici HTTP appropriati, facilitando così la comunicazione tra frontend e backend.

Inoltre, insieme alle librerie standard, nel progetto sono presenti librerie di terze parti come:

- `"github.com/golang-jwt/jwt/v5"` per la gestione dei **token JWT**
- `"github.com/joho/godotenv"` per il **caricamento delle variabili d'ambiente**
- `"github.com/lib/pq"` come **driver PostgreSQL**

Il Backend realizzato espone un set di **API REST** che garantiscono le principali funzionalità dell'applicazione, tra cui:

- Autenticazione degli utenti (registrazione, login)
- Gestione dei veicoli (inserimento, modifica, eliminazione)
- Gestione delle prenotazioni
- Profilazione e aggiornamento dell'utente autenticato
- Comunicazione con il modulo Python per ottenere statistiche

3.2.1. Struttura del backend e dettagli implementativi

Il backend di **CarRentalHub** è strutturato secondo un'architettura modulare che segue i principi della separazione delle responsabilità. Ogni *package* ha un ruolo specifico nella gestione delle richieste REST, del flusso dei dati e della logica applicativa.

Di seguito una descrizione dettagliata delle componenti principali:

- **Controllers (/controllers)**

Ogni controller espone gli endpoint HTTP appartenenti a un determinato dominio funzionale e smista le chiamate in entrata verso i service:

- ***auth_controller.go***: gestisce la registrazione e il login degli utenti. Al login, genera un token JWT, richiamando l'helper "*jwt_helper.go*". Le func presenti in questo controller sono "Register" e "Login".
- ***user_controller.go***: consente all'utente autenticato di aggiornare o eliminare il proprio profilo, rispettivamente tramite le funzioni "UpdateCurrentUser" e "DeleteCurrentUser".
- ***vehicle_controller.go***: le principali funzionalità accessibili solo dagli admin, permettono di creare, modificare e rimuovere veicoli dal parco auto, tramite le func "CreateVehicle", "UpdateVehicle", "DeleteVehicle". Mentre per i customer è possibile visualizzare tutti i veicoli, o un veicolo specifico tramite id, oppure di recuperare tutte le auto disponibili, nel range di date se specificato, per effettuare successivamente una prenotazione. Tutto ciò è possibile, tramite le func "GetAllVehicles", "GetVehicleById", "GetAvailableVehicles".
- ***booking_controller.go***: consente ai customer di creare nuove prenotazioni e visualizzare la propria cronologia, mentre agli admin consente di visualizzare tutte le prenotazioni effettuate tramite sistema, modificare e cancellare prenotazioni. Le func che permettono tutto ciò sono "GetAllBookings", "CreateBooking", "UpdateBooking".

- **Services (/services)**

Contengono la logica di business. Validano i dati ricevuti dai controller, applicano

eventuali regole aziendali e interagiscono con i repository.

Ad esempio:

- ***auth_service.go***: la funzione “RegisterUser” permette di creare un nuovo utente e applica tutti i controlli per verificare che lo username e la mail non siano già stati utilizzati e ritorna un DTO contenente le info del nuovo utente a meno della password. La funzione “AuthenticateUser”, richiamata dal controller quando si effettua il login, consente di recuperare l’utente dal db, tramite username o email, e di generare un token jwt in base al ruolo dell’utente che si sta loggando. La funzione ritorna un messaggio di login effettuato, il DTO contenente le info dell’utente e il token associato.
 - ***booking_service.go***: applica la logica per recuperare, creare, modificare ed eliminare le prenotazioni, utilizzando sia una cache per velocizzare il processo di recupero dati piuttosto che effettuare subito una chiamata al db. Inoltre, la gestione della concorrenza è gestita tramite Goroutine ed un canale globale. Le funzioni presenti in questo service sono: “GetAllBookings”, “CreateBooking”, “UpdateBooking”, “DeleteBooking”. La funzione utilizzata per modificare una prenotazione effettua un primo controllo per verificare che l’auto sia ancora disponibile nelle nuove date inserite. La funzione di creazione di una nuova prenotazione ritorna al frontend un oggetto BookingDTO che aggiunge oltre ai cmapi presenti in booking anche il totale da pagare e l’username dell’utente che sta effettuando la prenotazione.
 - ***user_service.go***: applica la logica per modificare ed eliminare un utente, tramite “UpdateUser” e “DeleteUser”.
 - ***vehicle_service.go***: applica la logica per recuperare tutti i veicoli, o i veicoli disponibili per una successiva prenotazione, per creare, modificare ed eliminare i veicoli. Le funzioni presenti sono: “CreateVehicle”, “UpdateVehicle”, “DeleteVehicle”, “GetAllVehicles”, “GetVehicleById”, “GetAvailableVehicles”
- **Repositories (/repositories)**

Racchiudono l’accesso diretto al database PostgreSQL. Utilizzano il package database/sql con query SQL parametrizzate. Tutte le operazioni di **creazione**, **aggiornamento** ed **eliminazione** vengono eseguite all'interno di **transazioni** database, garantendo la consistenza dei dati: in caso di errore durante l'esecuzione, viene effettuato automaticamente un **rollback**, evitando di lasciare il database in uno stato incoerente o parziale.

 - ***auth_repository.go***: “FindByEmail”, “FindByUsername” e “SaveUser” eseguono rispettivamente le query per recuperare un utente dalla tabella user tramite email o username e per memorizzare un nuovo utente in db.

- **booking_repository.go**: vengono eseguite le query per recuperare, creare, modificare ed eliminare una prenotazione dal database. “GetAllBookings” differenzia la query eseguita in base al ruolo dell’user che la richiede, se admin consente di recuperare tutte le prenotazioni esistenti, mentre per un customer è possibile recuperare solo le proprie prenotazioni, viene ritornato un array di bookings. “IsVehicleAvailable” esegue una query verso il db per verificare la disponibilità del veicolo selezionato, nelle date richieste. “CreateBooking” esegue la query per inserire una nuova prenotazione nel database ed esegue anche le query per recuperare l’username dell’utente che effettua la prenotazione ed il prezzo del veicolo per poi calcolare il prezzo totale in base ai giorni. “UpdateBooking” e “DeleteBooking” eseguono le query per aggiornare l’entry del database o per eliminarla.
- **user_repository.go**: “FindUserById”, “UpdateFullUser” e “DeleteUserById” eseguono rispettivamente le query per recuperare un utente tramite id, modificare i campi di un utente ed eliminare un utente dalla tabella.
- **vehicle_repository.go**: vengono eseguite le query per recuperare, creare, aggiornare o eliminare un veicolo dal database. “GetAllVehicles”, “GetVehicleById”, “GetAvailableVehicles” consentono rispettivamente di recuperare tutti i veicoli, un veicolo tramite id e di mostrare gli autosaloni che hanno dei veicoli disponibili nel range di date specificato, se le date non sono specificate allora vengono mostrati tutti gli autosaloni con tutti i veicoli posseduti. “CreateVehicle”, “UpdateVehicle”, “DeleteVehicle” consentono di eseguire le query per creare, aggiornare ed eliminare un veicolo dal database.
- **Models (/models)**
 Contengono le definizioni delle strutture dati principali del sistema. Ogni struct rappresenta una tabella del database. Mentre DTO rappresenta un oggetto trasferito tra i vari moduli.
 - **Booking**: *ID, UserID, VehicleID, StartDate, EndDate, CreatedAt, UpdatedAt*
 - **BookingDTO**: *UserID, Username, VehicleID, VehicleModel, StartDate, EndDate, CreatedAt, UpdatedAt, TotalPrice*.
 Oggetto ritornato alla creazione di una nuova prenotazione così da mostrare tutte le info sulla prenotazione, incluso l’username dell’utente che l’ha effettuata e il prezzo totale ottenuto dal prodotto tra il costo giornaliero del veicolo e i giorni di prenotazione.
 - **CarShowroom**: *ID, Name, Location, Latitude, Longitude*
 - **CarShowroomVehicleDTO**: *ID, Name, Location, Latitude, Longitude, Vehicles*.
 Oggetto ritornato alla richiesta di recupero di tutti i veicoli disponibili, così da

permettere all'utente di sapere qual è la lista di veicoli disponibili rispetto alle date indicate nella richiesta e in quale autosalone sono presenti.

- **User:** *ID, Username, Email, Password, Region, Role*
 - **UserLoginResponseDTO:** *ID, Username, Email, Region, Role*
Oggetto ritornato al login di un utente, insieme ad un token utile per le successive richieste. La password non è ritornata per aumentare la sicurezza e non inviare dati sensibili in chiaro.
 - **UserUpdateRequestDTO:** *Username, Email, Region, Password, NewPassword*
Oggetto ricevuto dal frontend quando un utente ha necessità di cambiare le informazioni del suo profilo, compreso la password utilizzata per accedere alla GUI. In questo caso è necessario inviare anche la password precedente per continuare l'operazione.
 - **Vehicle:** *ID, Model, Category, Price, CarShowroomID*
 - **VehicleDTO:** *ID, Model, Category, Price, CarShowroomID, CarshowroomName*.
Oggetto ritornato alla richiesta di recupero di tutti i veicoli presenti nel sistema, così da mostrare oltre le informazioni sul veicolo anche le informazioni principali sull'autosalone in cui si trovano.
- **Middleware (/middleware)**
Il middleware principale è **AuthMiddleware**, che intercetta le richieste protette, verifica la validità del JWT token e inserisce l'identificativo e il ruolo dell'utente nel context. Questo consente ai controller di sapere "chi sta chiamando" l'endpoint, e con che permessi, così da differenziare il ruolo dell'admin da quello del customer per le specifiche API.
 - **Routes (/routes)**
Qui vengono definiti i gruppi di route, con il prefisso **"/api/v1"** al quale viene aggiunto poi il dominio di appartenenza delle api, quindi **"/auth"**, per le richieste di autenticazione, **"/user"**, per le richieste relative alla modifica ed eliminazione di un utente, **"/vehicle"**, per le richieste relative ai veicoli, **"/booking"**, per le richieste relative alle prenotazioni. Ogni gruppo è protetto, dove necessario, dal middleware di autenticazione. Le route sono inizializzate nel main.go.
 - **Helpers (/helpers)**
Utility comuni, come la generazione del token JWT tramite la funzione **"GenerateToken"**, utilizzando l'algoritmo Hash256. Mentre, la funzione **"ValidateJWT"** consente di validare il token, facendo il parsing con i Claims e verificando che il token non sia valido, scaduto o malformato.

Questa struttura rende il **Backend** facilmente estendibile, testabile e scalabile. L'approccio modulare consente inoltre di isolare i componenti per facilitare la manutenzione e l'evoluzione futura del progetto.

3.2.2. Sicurezza

L'autenticazione è gestita tramite **JWT (JSON Web Token)**. Il token viene generato al momento del login e allegato alle richieste successive nella header Authorization. Il middleware AuthMiddleware verifica la validità del token e permette l'accesso solo agli endpoint autorizzati.

Sono definiti due ruoli:

- **customer:** utente che può prenotare veicoli e gestire il proprio profilo
- **admin:** utente con permessi estesi (gestione veicoli, gestione delle prenotazioni, accesso alle statistiche)

Ogni route ha un middleware in concatenazione che si occupa di verificare il token e contestualmente il ruolo dell'utente, così da garantire che solo l'utente con il ruolo corretto possa effettuare la funzionalità richiesta, evitando possibili incongruenze tra ruolo e funzione.

3.2.3. Database

Il Backend, come detto, si connette ad un database **PostgreSQL** il quale memorizza:

- **Utenti:** tabella "users".
- **Veicoli:** tabella "vehicles".
- **Prenotazioni:** tabella "bookings".
- **Autosaloni:** tabella "car_showrooms".

Le query sono eseguite tramite il pacchetto '*database/sq*' e sono parametrizzate per evitare SQL injection.

Le credenziali di accesso al db sono caricate attraverso un file di configurazione '.env', ovviamente non tracciato.

3.2.4. Endpoints

Tutte le API REST esposte sono organizzate per gruppi funzionali, ognuno dei quali è accessibile tramite il prefisso *"/api/v1/"*. A questo si aggiungono i path specifici per ciascun gruppo logico:

- */auth*: per le operazioni di registrazione e login.
- */user*: per la gestione dell'utente autenticato (profilo, cancellazione, aggiornamento)
- */vehicle*: per la gestione dei veicoli (visibile solo agli admin)

- */booking*: per la creazione, visualizzazione e gestione delle prenotazioni

DOMINIO	METODO	ENDPOINT	DESCRIZIONE
Auth	POST	/api/v1/auth/register	Permette la registrazione di un nuovo utente.
Auth	POST	/api/v1/auth/login	Permette il login di un utente (customer o admin) e restituisce un JWT.
User	PUT	/api/v1/user/update/me	Aggiorna i dati del profilo dell'utente autenticato.
User	DELETE	/api/v1/user/delete/me	Elimina l'account dell'utente autenticato.
Vehicle	GET	/api/v1/vehicle/getAll	Permette di recuperare tutti i veicoli presenti nei vari autosaloni.
Vehicle	GET	/api/v1/vehicle/getById/:id	Permette di recuperare uno specifico veicolo.
Vehicle	GET	/api/v1/vehicle/getAllAvailable	Permette di recuperare tutti i veicoli disponibili nel range di date passati come parametri.
Vehicle	POST	/api/v1/vehicle/new	Permette ad un admin di creare un nuovo veicolo.
Vehicle	PUT	/api/v1/vehicle/update/:id	Permette ad un admin di modificare un veicolo già esistente, identificato dall'id inviato nella richiesta.
Vehicle	DELETE	/api/v1/vehicle/delete/:id	Permette ad un admin di eliminare un veicolo esistente, identificato dall'id inviato nella richiesta.
Booking	POST	/api/v1/booking/new	Permette ad un customer di prenotare un veicolo.
Booking	GET	/api/v1/booking/getAll	Permette ad un customer di

			vedere lo storico delle proprie prenotazioni, mentre ad un admin di vedere lo storico delle prenotazioni di tutti i customer.
Booking	PUT	/api/v1/booking/updated/:id	Permette ad un admin di modificare una prenotazione già esistente, identificata dall'identificativo inviato come parametro.
Booking	DELETE	/api/v1/booking/delete/:id	Permette ad un admin di eliminare una prenotazione già esistente, identificata dall'identificativo inviato come parametro.

3.2.5. Gestione della concorrenza e parallelizzazione

Uno dei punti di forza del linguaggio **Go** è la gestione della concorrenza tramite le cosiddette **goroutine**, un meccanismo semplice, ma molto potente per eseguire più operazioni in parallelo.

Una **goroutine** è un'unità di esecuzione concorrente molto leggera, simile a un thread ma con un'occupazione di memoria e overhead ridotti. In Go, è possibile avviare una goroutine anteponendo la parola chiave **go** davanti a una funzione, come mostrato di seguito:

`“go processBookingRequests()”`.

Questa istruzione fa sì che *processBookingRequest* venga eseguita **in parallelo** rispetto al flusso principale del programma. Ogni goroutine viene gestita dal runtime Go e può essere eseguita in contemporanea con altre goroutine. Per coordinare più goroutine e gestire la comunicazione tra esse, Go mette a disposizione i **canali** (chan), che permettono di scambiarsi dati in modo sicuro.

In **CarRentalHub** le goroutine sono fondamentali per la gestione corretta e ottimale della concorrenza nella prenotazione dei veicoli.

Il problema affrontato è del tipo: due utenti potrebbero tentare di prenotare lo stesso veicolo per date che si sovrappongono, esattamente nello stesso istante, quando questo risulta ancora disponibile per entrambi. Senza un meccanismo di sincronizzazione, ciò potrebbe portare a una race condition e alla sovrapposizione delle prenotazioni, compromettendo la coerenza dei dati. Per evitare ciò, si è adottato un **approccio** con goroutine più canale condiviso. In particolare, nel file `booking_service.go`:

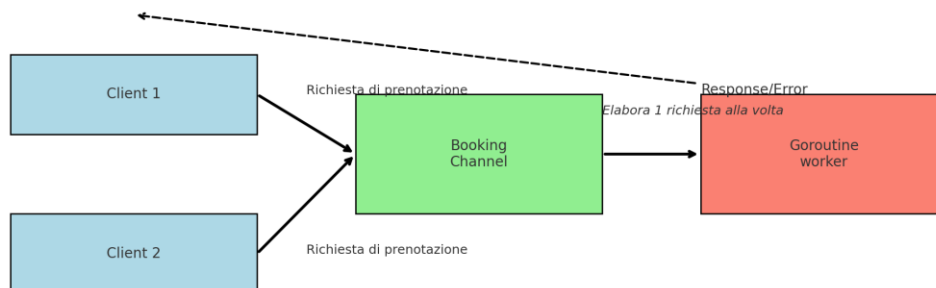
- Una **goroutine worker** viene avviata all'avvio del sistema tramite `init()` “go `processBookingRequests()`”
- Tutte le richieste di prenotazione vengono **inviare su un canale tramite** “`bookingChannel <- BookingRequest{...}`”
- La goroutine worker legge sequenzialmente dal canale e processa **una richiesta alla volta**, garantendo l'accesso esclusivo alla logica di booking:

```
for req := range bookingChannel {

    createdBooking, err := CreateBooking(&req.Booking)

... }
```

- La funzione `RequestBooking(...)` nel controller scrive sul canale e attende la risposta tramite canali dedicati “`response`” e “`errorChan`”, offrendo un'interfaccia sincrona verso l'esterno, ma con gestione concorrente interna sicura.



3.2.6. Implementazione della cache

Nel backend di **CarRentalHub** è stato implementato un sistema di **cache in memoria personalizzata**, con l'obiettivo di **ottimizzare le prestazioni** e **ridurre il numero di query al database PostgreSQL**, specialmente per le operazioni più frequenti come la lettura delle prenotazioni o dei veicoli disponibili.

Una **cache** è una struttura temporanea che memorizza copie di dati frequentemente richiesti, per evitare di doverli ricalcolare o rileggere da fonti lente come il database. In questo caso, si

tratta di una cache mantenuta nella RAM del server, che consente accessi rapidissimi. Il modulo *cache/* definisce una struttura Cache composta da:

- una mappa concorrente (`sync.Map`) per gestire **accessi paralleli sicuri**
- un campo `ttl` (time-to-live) per definire **quanto dura ogni entry** prima di essere considerata scaduta

L'inserimento dei dati in cache e la lettura di questi è stato implementato rispettivamente dalle funzioni `Set` e `Get`. Nel caso di lettura dei dati dalla cache, la validità dell'entry viene verificata confrontando il tempo attuale con `expiredAt`. Se l'elemento è scaduto, viene eliminato automaticamente. Alla creazione di una nuova cache (`NewCache(ttl)`), viene avviata **una goroutine di cleanup** che periodicamente (ogni `ttl / 2`) controlla e rimuove le entry scadute: `"go c.cleanupExpiredEntries()"`, questa goroutine scorre tutte le chiavi memorizzate e le elimina se risultano scadute, mantenendo la cache "pulita" e aggiornata. Le istanze della cache vengono inizializzate in `cache_manager.go`, con un TTL di 30 secondi, precisamente sono definite `"BookingCache"` e `"VehicleCache"` utilizzate una per memorizzare temporaneamente le prenotazioni effettuate e una per il caching dei veicoli per ridurre le query al db durante le ricerche. Inoltre, alla creazione di una nuova prenotazione o di un nuovo veicolo, oppure di un aggiornamento, entrambe le cache vengono invalidate, garantendo così che la cache non conserva dati obsoleti dopo un'operazione di scrittura. I vantaggi apportati da questa soluzione sono un accesso rapido ai dati frequentemente richiesti, una riduzione del carico sul database, oltre che migliorare la scalabilità.

3.3. Statistics Module [Python]

Il modulo `Statistics` rappresenta il componente dedicato all'analisi dei dati e alla generazione di metriche utili alla valutazione delle performance del sistema, sviluppato in **Python**. Il modulo è containerizzato separatamente e interagisce con il database PostgreSQL condiviso con il Backend Go, dal quale estrae i dati per produrre analisi aggregate e visualizzazioni statistiche. Per la realizzazione di questo modulo è stato scelto **Flask**, un web framework scritto in Python molto diffuso. Flask fornisce tutti gli strumenti essenziali per sviluppare applicazioni web o API REST leggere e modulari, lasciando al programmatore la libertà di strutturare il progetto secondo le proprie esigenze.

Nel progetto `CarRentalHub`, Flask è stato utilizzato per:

- **Definire routing ed endpoint REST** in maniera semplice e diretta, grazie al decoratore `@app.route`;
- **Gestire richieste HTTP parametrizzate** (ad esempio per filtrare dati in base a `start_date` e `end_date`);
- **Restituire risposte sia in formato JSON che immagine**, integrandosi con librerie come `matplotlib` per la generazione di grafici;

Il server Flask viene avviato nel file `app.py`, configurato per l'esecuzione all'interno di un container Docker, e rimane in ascolto sulla porta 8085, pronto a ricevere richieste da parte del Frontend o del Backend Go.

Le principali funzionalità offerte dal modulo `statistics` includono:

- **Statistiche sugli utenti:** top utenti per numero di prenotazioni, distribuzione geografica, spesa totale per singolo utente.
- **Statistiche sui veicoli:** percentuale di disponibilità, veicoli più richiesti.
- **Statistiche sulle prenotazioni:** trend dei guadagni, andamento temporale, ricavo per veicolo.

Le richieste verso il modulo sono esposte tramite un set di endpoint REST che accettano parametri temporali (`start_date`, `end_date`) per filtrare dinamicamente i dati. La generazione dei grafici avviene mediante la libreria **Matplotlib**, mentre la risposta viene restituita in formato immagine (PNG) oppure JSON a seconda del tipo di metrica.

3.3.1. Struttura e dettagli implementativi

Il modulo `Stats` di `CarRentalHub` è progettato secondo un'architettura modulare che rispetta i principi della separazione delle responsabilità. Il codice è organizzato in moduli distinti, ognuno dei quali ha un ruolo specifico nella gestione delle richieste REST, nella logica di elaborazione e nell'accesso ai dati.

Di seguito una descrizione dettagliata dei vari moduli:

- **Controllers (/controllers)**
Ogni controller espone gli endpoint REST relativi ad uno specifico dominio statistico e smista le richieste in ingresso verso i rispettivi service.
 - **booking_controller.py:** gestisce le statistiche relative alle prenotazioni. La funzione `"revenue_trend()"` esposta dall'endpoint `"/revenue/trend"` calcola i guadagni ottenuti dalle varie prenotazioni nel periodo temporale specificato. Mentre tramite la funzione `"revenue_for_vehicle()"` esposta dall'endpoint `"/revenueforvehicle/trend"` restituisce il ricavo complessivo per ogni veicolo. La funzione `"booking_trend()"` esposta dall'endpoint `"/numbookings/trend"` restituisce il numero di prenotazioni effettuate nel periodo specificato.
 - **user_controller.py:** gestisce le statistiche relative agli utenti, permettendo di recuperare info sui tre utenti con il maggior numero di prenotazioni tramite la funzione `"user_top3()"` richiamata dall'endpoint `"/top3"`. La funzione `"user_regions_distribution()"` all'endpoint `"/regions"` mostra la distribuzione geografica dei clienti che usano il software. Inoltre, è possibile recuperare informazioni come il numero di prenotazione e la spesa totale per uno specifico utente, grazie alla funzione `"user_info(user_id)"` richiamata al path `"/info/<int:user_id>"`.
 - **vehicle_controller.py:** fornisce le statistiche relative alle prenotazioni e alla disponibilità dei veicoli. Tramite la funzione `"vehicle_stats_availability()"` esposta al path `"/availability"` è possibile recuperare un grafico con le

percentuali dei veicoli disponibili in un dato periodo. Mentre la funzione `"vehicle_stats_top5()"` al path `"/top5"` ritorna l'elenco dei cinque veicoli con più prenotazioni.

- **Services (/service)**

I moduli all'interno di questa directory contengono la logica applicativa. Elaborano i dati, applicano le regole statistiche e interagiscono con i repository per l'accesso ai dati. I dati possono essere ritornati o in formato JSON o come immagine utilizzando il formato PNG.

- **booking_service.py**: presenta le funzioni, richiamate dal controller, `"generate_revenue_trend_plot(start_date=None, end_date=None)"`, `"generate_booking_trend_plot(start_date=None, end_date=None)"`, `"generate_revenue_per_vehicle_plot(start_date=None, end_date=None)"` utili rispettivamente per calcolare trend economici, calcolare il numero di prenotazioni e calcolare i ricavi per veicolo. Le tre funzioni processano i dati recuperati dal db tramite query e restituiscono delle immagini in formato .png.
- **user_service.py**: aggrega i dati relativi agli utenti, come top 3 utenti con il numero maggiore di prenotazioni, distribuzione geografica degli utenti e informazioni su singolo utente, rispettivamente grazie alle funzioni `"generate_top_users_plot()"`, `"generate_users_by_region_plot()"`, `"get_user_info(user_id)"`. Le prime due funzioni elaborano i dati e generano una risposta sotto forma di immagine in formato.png, mentre l'ultima funzione restituisce un JSON.
- **vehicle_service.py**: elabora le statistiche sui veicoli, come disponibilità e top 5 veicoli con più prenotazioni, richiamando le funzioni `"generate_availability_plot(start_date=None, end_date=None)"`, `"generate_top_vehicles_plot(start_date=None, end_date=None)"` e ritornando in entrambi i casi delle immagini in formato .png.

- **Repositories (/repositories)**

I moduli presenti in questa directory gestiscono l'accesso diretto al database PostgreSQL, eseguendo query SQL parametrizzate e restituendo i dati ai relativi service.

- **booking_repository.py**: in questo modulo sono presenti tre funzioni che permettono di ottenere statistiche sull'andamento economico e operativo del sistema di noleggio. La funzione `"get_revenue_trend(start_date=None, end_date=None)"` restituisce l'andamento giornaliero dei ricavi, cioè quanto è stato guadagnato per ogni giorno in cui sono presenti prenotazioni. Se i parametri `start_date` e `end_date` sono stati forniti la query viene filtrata per questo range temporale, se non sono stati forniti, invece, vengono considerate tutte le prenotazioni presenti nel db. La funzione `"get_booking_trend(start_date=None, end_date=None)"` restituisce il numero di prenotazioni per giorno, basandosi sulla data di inizio di ciascuna prenotazione, se vengono forniti i parametri l'andamento restituito si basa solo sul range temporale indicato, altrimenti viene restituito l'andamento completo di tutte le prenotazioni. La funzione `"get_revenue_per_vehicle(start_date=None, end_date=None)"` calcola quanto ha guadagnato ciascun veicolo, in un certo

periodo o su tutta la storia a seconda del passaggio dei parametri `start_date` e `end_date` o meno.

- **`user_repository.py`**: sono presenti le funzioni richiamate per ottenere statistiche sugli utenti di tipo "customer". La funzione `"get_top_3_users()"` restituisce i tre utenti con più prenotazioni nel sistema. La funzione `"get_users_by_region()"` restituisce la distribuzione geografica degli utenti, ovvero indicat quanti utenti utilizzano il sistema per ogni regione. Mentre la funzione `"get_user_statistics(user_id)"` restituisce le statistiche di un singolo utente, in particolare il numero totale di prenotazioni e l'importo totale speso.
- **`vehicle_repository.py`**: la funzione `"get_vehicle_availability(start_date=None, end_date=None)"` esegue la query per recuperare i veicoli disponibili, differenziando le query sulla base delle date. Se le date sono passate allora la query eseguita filtra i risultati sulla base del range temporale indicato, altrimenti viene eseguita una query recuperando la disponibilità odierna, usando `"CURRENT_DATE"` come riferimento. Mentre la funzione `"get_top_5_vehicles(strat_date=None, end_date=None)"` esegue la query per ottenere i 5 veicoli più prenotati. Se i parametri `start_date` e `end_date` vengono forniti, la funzione filtra le prenotazioni in quel range temporale, mentre se non sono forniti, la query considera tutte le prenotazioni presenti nel sistema.
- **Utils (/utils)**
 - **`date_utils.py`**: questo modulo gestisce la validazione del formato e il parsing dei parametri `start_date` e `end_date`, restituendo un range temporale corretto da usare nelle query SQL. Se i parametri sono errati o solo uno dei due è mancante, ritorna un messaggio di errore formattato per una risposta Flask. Mentre se nessuno delle due date è fornita vuol dire che non si vuole applicare nessun filtro temporale. La funzione richiamata in questo modulo è `"validate_and_get_date_range(start_date, end_date)"`
- **Configurazione e ingresso**
 - **`app.py`**: rappresenta il punto di ingresso del modulo. Inizializza il server Flask, importa e registra i controller, avviando l'app sulla porta 8085.
 - **`config.py`**: carica le variabili d'ambiente da `.env` (es. credenziali del DB).
 - **`db.py`**: crea e gestisce la connessione al database PostgreSQL, utilizzata in tutti i repository.

Questa struttura modulare consente una facile estendibilità del progetto, semplifica la manutenzione del codice e garantisce un'organizzazione coerente anche nel contesto Python.

4. Avvio applicazione

4.1. Modulo Frontend

Il modulo Frontend specifica come target il framework .NET 9.0 nel file di progetto. Pertanto, per eseguire correttamente l'applicazione è necessario avere installata questa versione di .NET. Le dipendenze vengono automaticamente risolte e installate durante la fase di build, grazie alle

direttive del file `.csproj`, che includono pacchetti come *Avalonia*, *CommunityToolkit.Mvvm*, *Mapsui* e i relativi moduli per la visualizzazione delle mappe. Il comando di build (`dotnet build`) si occupa anche di scaricare i pacchetti mancanti e configurare l'ambiente di runtime.

4.2. Moduli Backend e Stats

I moduli Backend e Stats sono definiti tramite un file **`docker-compose.yaml`**, che gestisce il deployment di tre container: il database PostgreSQL (db), il backend scritto in Go (backend) e il modulo in Python per le statistiche (stats).

È necessario installare la versione più recente di Docker per poter eseguire i due moduli. Per il corretto funzionamento dell'applicazione, è necessario creare un file `.env` nella directory Backend contenente le variabili d'ambiente, come le credenziali del database (DB_USER, DB_PASSWORD, DB_HOST) e il nome del database (DB_NAME).

Il servizio db utilizza l'immagine ufficiale di PostgreSQL e monta un volume persistente per mantenere i dati (pgdata). Inoltre, viene configurato un healthcheck per verificare la disponibilità del database prima dell'avvio dei moduli dipendenti.

Il servizio backend si basa su un'immagine personalizzata, costruita tramite il Dockerfile specifico presente nella directory del modulo Go.

Il servizio stats, scritto in Python e basato su Flask, viene containerizzato separatamente e dipende anch'esso dal database. Questo modulo è configurato per l'avvio automatico in modalità sviluppo, con variabili d'ambiente specifiche.

4.3. Avvio del Sistema

Per avviare il progetto localmente assicurarsi che le porte 5432, 8085 e 5005 siano disponibili perché utilizzate rispettivamente dal modulo Backend, Stats e PostgreSQL.

Dopo aver configurato correttamente il file `.env`, bisogna prima avviare Docker e navigare nella directory `./docker`. Da lì, eseguire il comando: **`docker-compose up --build -d`**.

Questo comando genera e avvia tutti i container, rispettando le dipendenze dichiarate. Durante l'avvio, i container backend e stats attendono il completamento dell'healthcheck del database prima di iniziare il loro ciclo operativo, garantendo così l'ordine corretto di inizializzazione.

Una volta che i container sono stati creati e avviati correttamente, accedere alla directory `./Frontend` e lanciare il comando: **`dotnet run`**.

Così facendo, l'applicazione Frontend viene compilata ed eseguita, aprendo una finestra interattiva con l'interfaccia utente. A questo punto, il sistema è pronto per l'uso.